Assignment 1 Report: Simple OpenGL Project

Group Report

May 17, 2026

1 Introduction

This report details the implementation of a simple OpenGL application. The project successfully loads a 3D model (Trophy Table) downloaded from a 3D asset provider, and renders it using custom vertex and fragment shaders. The application features user controls to modify matrices, lighting conditions, and materials.

2 Implementation Details

2.1 Vertex and Fragment Shaders (Phong Model)

The application implements the standard Phong reflection model directly within the `fragment.glsl` file. The shader calculates three main components of lighting: Ambient, Diffuse, and Specular.

- **Ambient Lighting:** Simulates indirect light bounding off surfaces so that no shadow is completely black.
- **Diffuse Lighting:** Simulates the directional impact of the light object, making parts facing the light brighter.
- **Specular (Phong) Lighting:** Simulates the bright spot/reflection of a light on objects.

Below is the snippet from the fragment shader calculating the Phong reflections:

```
vec3 norm = normalize(Normal);
vec3 viewDir = normalize(viewPos - FragPos);

// Diffuse
float diff = max(dot(norm, lightDir), 0.0);
diffuse += intensity * diff * color * material.diffuse * texColor;

// Specular (Phong)
vec3 reflectDir = reflect(-lightDir, norm);
float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);
specular += intensity * spec * color * specMask;
```

Different parts of the mesh are given different material characteristics (like glass, metal, and wood) by passing different `material.shininess` and `material.alpha` values via uniforms. The code also implements logic to discard pixels for transparent logic separating opaque passes and blending passes.

2.2 Loading a 3D Model (VAO & VBO)

The project loads complex `.obj` files using the Assimp (Asset Importer) Library. In the codebase, the `Model` and `Mesh` classes handle the abstraction. Once vertex data (Position, Normal, Texture Coordinates) is parsed, it is securely bound to the GPU leveraging Vertex Array Objects (VAO) and Vertex Buffer Objects (VBO).

```
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO);

glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(Vertex),
             &vertices[0], GL_STATIC_DRAW);
```

This approach ensures the pipeline only copies data to the graphics card once, which is highly efficient. In the rendering loop, `glBindVertexArray(VAO)` is called before invoking `glDrawElements`.

2.3 Model, View, and Projection Matrices Setup

The 3D space transformations are thoroughly demonstrated with interactive key-bindings:

- **Model Matrix:** Controls the position, scale, and rotation of the object. By pressing the **'R'** key, an auto-rotation feature modifies the model matrix over time frame-by-frame using `glm::rotate`.
- **View Matrix:** Simulates a moving camera. Users can press **'V'** to cycle between three defined camera offsets (Cinematic, Side, and Top perspectives) achieved using `glm::lookAt()`.
- **Projection Matrix:** Simulates the lens of the camera. Pressing **'P'** toggles the application between a perspective projection (simulating human eyes via `glm::perspective()`) and an orthographic projection (ignoring depth variance via `glm::ortho()`).

3 Screenshots

(Please insert the final screenshots of the running app here. Included pictures should demonstrate the Orthographic vs Perspective projections, Textures enabled/disabled, and different view angles.)